

Entrega 5 - Regresion Logistica

Rodrigo Mansilla, Javier Chen

2025-02-25

##Creación de las variables dicotomicas

Table 1: Distribución de PriceCat en train_data

Var1	Freq
Economicas	256
Intermedias	509
Caras	259

Table 2: Distribución de Es_Economica en train_data

Var1	Freq
0	768
1	256

Table 3: Distribución de Es_Intermedia en train_data

Var1	Freq
0	515
1	509

Table 4: Distribución de Es_Cara en train_data

Var1	Freq
0	765
1	259

Se crearon tres variables dicotómicas que indican la pertenencia o no a cada categoría:

Es_Económica (1 si la vivienda es económica, 0 si no lo es)

Es_Intermedia (1 si la vivienda es intermedia, 0 si no lo es)

Es_Cara (1 si la vivienda es cara, 0 si no lo es) ## Modelo de regresión logística El modelo propuesto incluye como predictores a las variables:

OverallQual (calidad global de la vivienda)

GrLivArea (superficie habitable sobre rasante)

YearBuilt (año de construcción de la vivienda)

Table 5: Resumen del Modelo Logístico

Line
Call: NULL
Coefficients: Estimate Std. Error z value Pr(> z) (Intercept) -3.3155 0.2530 -13.107 < 2e-16 *** OverallQual 2.1529 0.2355 9.142 < 2e-16 *** GrLivArea 2.4332 0.2410 10.097 < 2e-16 *** YearBuilt 0.6520 0.1567 4.161 3.17e-05 ***
— Signif. codes: 0 ‘ ’ 0.001 ’ ’ 0.01 ’ ’ 0.05 ‘ ’ 0.1 ’ ’ 1
(Dispersion parameter for binomial family taken to be 1)
Null deviance: 1158.21 on 1023 degrees of freedom Residual deviance: 436.63 on 1020 degrees of freedom AIC: 444.63
Number of Fisher Scoring iterations: 7

Se observa el resumen de la regresión logística:

- Intercept = -3.3155 ($p < 2e-16$)
- OverallQual = 2.1529 ($p < 2e-16$)
- GrLivArea = 2.4332 ($p < 2e-16$)
- YearBuilt = 0.6520 ($p = 3.17e-05$)

Todas las variables son estadísticamente significativas al nivel de $p < 0.001$, lo que indica que tienen un efecto importante en la probabilidad de que una vivienda sea clasificada como “cara”, la probabilidad de ser “cara” es muy baja.

- Null deviance: 1158.21
- Residual deviance: 436.63
- AIC: 444.63

La gran reducción de la deviance indica que el modelo está ajustando notablemente mejor que un modelo nulo sin predictores.

Interpretación de los coeficientes

- OverallQual: Incrementar en 1 unidad esta variable multiplica las probabilidades de que una vivienda sea cara por $\exp(2.1529)$.
- GrLivArea: Cada unidad adicional de GrLivArea aumenta las probabilidades de ser cara por $\exp(2.4332)$.
- YearBuilt: Cada año adicional de antigüedad (más reciente) se asocia con un factor multiplicativo de $\exp(0.6520)$ en las probabilidades de ser cara.

Es importante revisar la escala real de cada predictor para interpretar correctamente el impacto absoluto. Sin embargo, el alto nivel de significancia de las variables confirma que las tres capturan factores relevantes para predecir si la vivienda se encuentra en la categoría de “caras”.

Predicción y evaluación del modelo

Table 6: Matriz de Confusión en Entrenamiento

Predicted	Actual	Freq
0	0	727
1	0	38
0	1	56
1	1	203

Table 7: Precisión del Modelo en Entrenamiento

Metrica	Valor
Precisión	0.9082031

Table 8: Sensibilidad del Modelo en Entrenamiento

Metrica	Valor
Sensibilidad	0.8423237

Table 9: Especificidad del Modelo en Entrenamiento

Metrica	Valor
Especificidad	0.9284802

Multicolinealidad

Table 10: Valores de VIF para el Modelo Logístico

Variable	VIF
OverallQual	1.183839
GrLivArea	1.274443
YearBuilt	1.380776

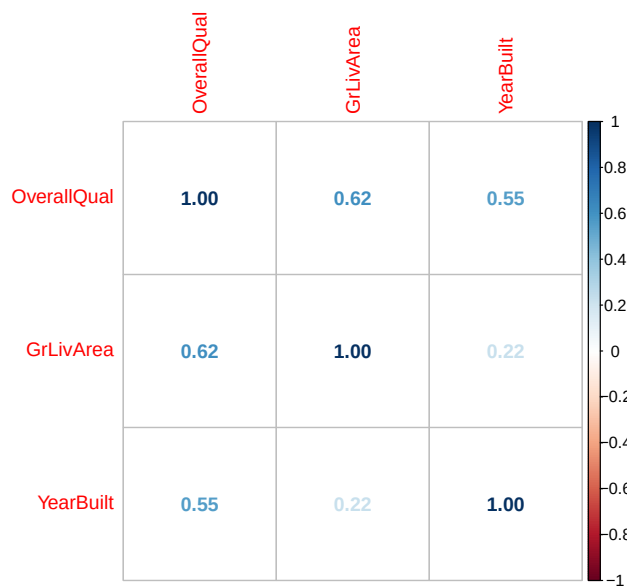
Variable	VIF
----------	-----

Un VIF inferior a 5 (incluso algunos autores toman 10) sugiere **ausencia de multicolinealidad severa**. En este caso, los valores están muy por debajo de 5, lo que indica que **no** hay problemas graves de multicolinealidad en las variables predictoras.

Matriz de correlación

Table 11: Matriz de Correlación de Predictores

	OverallQual	GrLivArea	YearBuilt
OverallQual	1.0000000	0.6185641	0.5527962
GrLivArea	0.6185641	1.0000000	0.2225015
YearBuilt	0.5527962	0.2225015	1.0000000



Si bien las correlaciones entre **OverallQual** y **GrLivArea** (0.62) y entre **OverallQual** y **YearBuilt** (0.55) son moderadas, no son lo suficientemente altas como para suponer un problema de colinealidad. Esto coincide con los bajos valores de VIF.

Predicción en el conjunto de prueba

Table 12: Matriz de Confusión en el Conjunto de Prueba

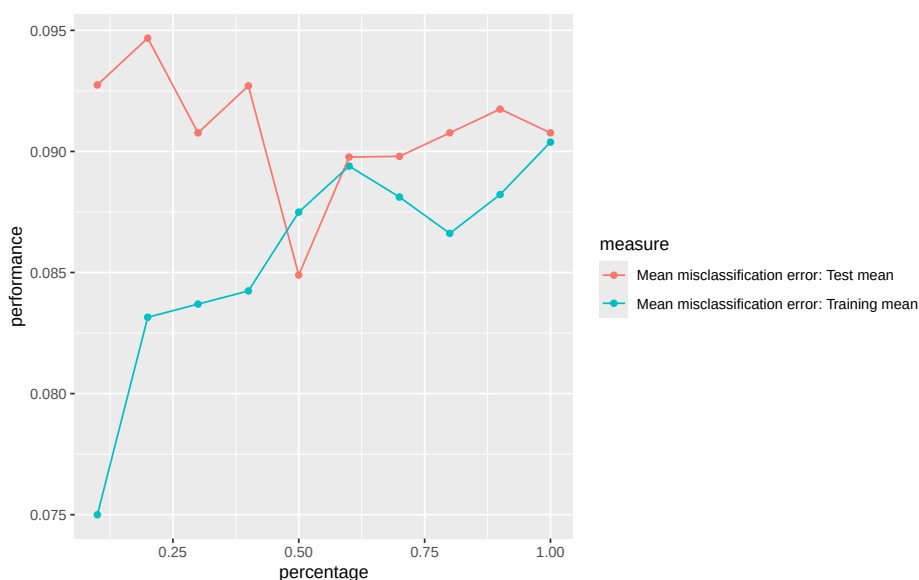
Predicted	Actual	Freq
0	0	306
1	0	22
0	1	23
1	1	85

Table 13: Métricas Globales del Modelo en el Conjunto de Prueba

Accuracy	Kappa	AccuracyLower	AccuracyUpper	AccuracyNull	AccuracyPValue	McnemarPValue
0.896789	0.7222065	0.8643385	0.9237163	0.7522936	0	1

Una exactitud de casi 90% en el conjunto de prueba indica que el modelo conserva un buen desempeño fuera de la muestra de entrenamiento. Además, el valor de **Kappa** de alrededor de 0.72 sugiere un acuerdo sustancial entre las predicciones y la realidad.

Curvas de Aprendizaje



Para determinar si existe sobreajuste, se comparan los resultados en entrenamiento y en prueba:

- **Entrenamiento**
- Precisión: 0.9082 (90.82%)
- **Prueba**
- Precisión: 0.8968 (89.68%)
-

La diferencia entre 90.82% (entrenamiento) y 89.68% (prueba) es relativamente pequeña (1.14 puntos porcentuales). Esto sugiere que **no hay un sobreajuste considerable**, pues el modelo rinde de manera bastante uniforme tanto en el conjunto de entrenamiento como en el de prueba.

El gráfico de las curvas de aprendizaje muestra la evolución del error de entrenamiento y de prueba a medida que aumenta el tamaño de los datos utilizados. Se observa que:

- **Alta variabilidad con pocos datos:** Al usar menos del 30% de los datos, el error en validación es alto y variable, reflejando la inestabilidad por la baja cantidad de información.
- **Incremento del error de entrenamiento:** Con más datos, el error de entrenamiento aumenta levemente, ya que el modelo ya no se sobreajusta tan fácilmente a muestras pequeñas.
- **Disminución y estabilización del error de validación:** Al aumentar el conjunto de entrenamiento, el error en validación desciende y se vuelve más consistente.
- **Convergencia de ambas curvas:** La diferencia entre el error en entrenamiento y en validación se reduce al final, lo que indica una buena capacidad de generalización y ausencia de sobreajuste significativo.
- **Mejora global:** El modelo se beneficia de tener más datos, lo que conduce a un comportamiento más estable y predecible en la clasificación.

Tuneo de Modelo

Table 14: Mejores Parámetros del Modelo Regularizado

	alpha	lambda
	7	0 0.0464159

Se usa una regularización Ridge con un grado de penalización aplicado para evitar coeficientes excesivamente grandes y, en consecuencia, reducir el sobreajuste. Buscamos optimizar la capacidad predictiva del modelo regularizado en comparación con el modelo base, encontrando un buen equilibrio entre sesgo y varianza.

Matriz de Confusión

Table 15: Matriz de Confusión - Modelo Base

Predicted	Actual	Freq
No	No	306
Sí	No	22
No	Sí	23
Sí	Sí	85

Table 16: Métricas de Evaluación - Modelo Base

	Métrica	Valor
Accuracy	Precisión	0.8967890
Sensitivity	Recall (Sensibilidad)	0.9329268
Specificity	Especificidad	0.7870370
F1	F1-Score	0.9315068

Interpretación del Modelo Base: - La accuracy de 89.7% indica un buen desempeño global en la clasificación.

- La alta sensibilidad 93.3% muestra que el modelo identifica correctamente la mayoría de las viviendas “caras”.

- La especificidad de 78.7% sugiere que, en el otro sentido, el modelo comete más errores al clasificar correctamente las viviendas que no son caras.
- En conjunto, estos resultados indican que el modelo base tiene un rendimiento robusto, pero con cierta tendencia a producir falsos positivos al etiquetar casas como “caras” cuando no lo son.

Medición del tiempo y comparacion

Table 17: Comparación de Tiempos de Ejecución (Notación Científica)

Modelo	Tiempo_Total
Base (glm)	0.0000e+00
Regularizado (glmnet)	2.0000e-02

El modelo regularizado muestra una pequeña sobrecarga temporal (0.02 segundos) respecto al modelo base, esta diferencia es marginal y justificada por el beneficio de incluir regularización y tuning, lo que puede mejorar la robustez del modelo a través de una mejor selección de hiperparámetros y el control del sobreajuste. El tiempo 0 de el modelo base es una aproximación a un tiempo demasiado pequeño.

Comparación de Modelos (Modelo Regularizado)

Table 18: Matriz de Confusión - Modelo Regularizado

Prediction	Reference	Freq
No	No	317
Sí	No	11
No	Sí	30
Sí	Sí	78

Table 19: Métricas de Evaluación - Modelo Regularizado

	Métrica	Valor
Accuracy	Precisión	0.9059633
Sensitivity	Recall (Sensibilidad)	0.9664634
Specificity	Especificidad	0.7222222
F1	F1-Score	0.9392593

Interpretación del Modelo Regularizado:

- EL accuracy mejora ligeramente hasta aproximadamente 90.6%, lo que implica una mejora global en la clasificación.
- La sensibilidad aumenta a 96.65%, lo que significa que el modelo regularizado es más efectivo en identificar correctamente las viviendas “caras”.
- La especificidad disminuye a 72.22%, indicando un incremento en el número de falsos positivos , casas que no son caras pero se clasifican como tales.

- El F1-Score elevado (0.9393) denota un buen balance entre precisión y sensibilidad, siendo levemente superior en comparación con el modelo base.

Discusión Global

Optimización del modelo: El tuneo con regularización ha permitido mejorar ligeramente la accuracy y, de forma más notable, la sensibilidad del modelo. El modelo regularizado es más efectivo en identificar viviendas caras, aunque a costa de una disminución en la especificidad.

Elección del modelo según la aplicación: La decisión entre utilizar el modelo base o el regularizado dependerá del costo asociado a los errores.

- Si se requiere maximizar la detección de viviendas caras (prioridad a la sensibilidad), el modelo regularizado es preferible.
- Si se valora más minimizar los falsos positivos (mejor especificidad), el modelo base podría ser más adecuado.

Balance Global: El leve incremento en el F1-Score del modelo regularizado sugiere que, en general, ofrece un balance más favorable entre precisión y sensibilidad, pese a su menor especificidad.

Modelo de Clasificación

Modelo Estandar

weights: 15 (8 variable)

initial value 1124.978984 iter 10 value 509.675531 final value 507.683971 converged

Table 20: Resumen del Modelo Estándar (multinom)

Line
Call: multinom(formula = model_formula, data = train_data)
Coefficients: (Intercept) OverallQual GrLivArea YearBuilt Intermedias 2.5555045 1.135590 1.364110 1.411923 Caras -0.6544799 3.272287 3.745155 1.993227
Std. Errors: (Intercept) OverallQual GrLivArea YearBuilt Intermedias 0.1998333 0.1864077 0.1552200 0.1491263 Caras 0.3180783 0.2972800 0.2877176 0.2134072
Residual Deviance: 1015.368 AIC: 1031.368

Table 21: Matriz de Confusión - Modelo Estándar

Prediction	Reference	Freq
Economicas	Economicas	88
Intermedias	Economicas	21
Caras	Economicas	0
Economicas	Intermedias	20
Intermedias	Intermedias	177
Caras	Intermedias	22
Economicas	Caras	0
Intermedias	Caras	21
Caras	Caras	87

Modelo con Validación Cruzada

weights: 15 (8 variable)

initial value 1011.821918 iter 10 value 461.347479 final value 460.204370 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 464.912073 final value 464.091170 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 461.351083 final value 460.208356 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 471.506914 final value 469.896600 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 475.165818 final value 473.610311 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 471.510591 final value 469.900406 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 457.220240 final value 453.122266 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 461.341959 final value 457.156007 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 457.224348 final value 453.126409 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 457.266876 final value 454.984026 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 461.044934 final value 459.048023 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 457.270673 final value 454.988202 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 453.270396 final value 451.069321 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 457.242044 final value 455.113309 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 453.274383 final value 451.073475 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 447.477195 final value 446.029798 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 451.187803 final value 450.232411 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 447.480950 final value 446.034118 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 461.046286 final value 459.737873 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 464.782039 final value 463.672848 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 461.050066 final value 459.741911 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 460.391356 final value 457.090283 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 463.794195 final value 461.095138 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 460.394810 final value 457.094395 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 464.774788 final value 462.733788 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 468.448688 final value 466.598182 converged # weights: 15 (8 variable)
 initial value 1012.920530 iter 10 value 464.778484 final value 462.737752 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 452.517760 final value 449.935399 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 456.068589 final value 454.140779 converged # weights: 15 (8 variable)
 initial value 1011.821918 iter 10 value 452.521358 final value 449.939726 converged # weights: 15 (8 variable)
 initial value 1124.978984 iter 10 value 514.084720 final value 511.669456 converged Resumen del Modelo con CV:

Table 22: Resumen del Modelo con Validación Cruzada (multinom)

Line
Call:
nnet::multinom(formula = .outcome ~ ., data = dat, decay = param\$decay)
Coefficients:
(Intercept) OverallQual GrLivArea YearBuilt
Intermedias 2.4976913 1.104336 1.331862 1.382221
Caras -0.6430377 3.200946 3.653611 1.945692
Std. Errors:
(Intercept) OverallQual GrLivArea YearBuilt
Intermedias 0.1955114 0.1841905 0.1529993 0.1467669
Caras 0.3104671 0.2926697 0.2825574 0.2102655
Residual Deviance: 1023.339
AIC: 1039.339

Matriz de Confusión - Modelo con CV:

Table 23: Matriz de Confusión - Modelo con CV

Prediction	Reference	Freq
Economicas	Economicas	88
Intermedias	Economicas	21
Caras	Economicas	0
Economicas	Intermedias	20
Intermedias	Intermedias	177
Caras	Intermedias	22
Economicas	Caras	0
Intermedias	Caras	21
Caras	Caras	87

La CV se utiliza para estimar mejor el error real y evitar sobreajuste, pero en este caso no propició un aumento en la accuracy final ni cambios drásticos en los coeficientes, posiblemente porque el dataset o la complejidad del modelo no dieron margen para un aumento sustancial en el desempeño.

Modelo con tuneo de hiperparámetros

Table 24: Resumen del Modelo Tuneado (glmnet)

Line
glmnet
1024 samples
3 predictor
3 classes: 'Economicas', 'Intermedias', 'Caras'
No pre-processing

Line				
Resampling: Cross-Validated (10 fold)				
Summary of sample sizes: 921, 921, 922, 922, 922, 922, ...				
Resampling results across tuning parameters:				
alpha	lambda	Accuracy	Kappa	
0.0	0.0001	0.7821150	0.6415131	
0.0	0.0112	0.7821150	0.6415131	
0.0	0.0223	0.7821150	0.6415131	
0.0	0.0334	0.7821055	0.6406238	
0.0	0.0445	0.7772606	0.6303914	
0.0	0.0556	0.7772796	0.6294086	
0.0	0.0667	0.7801923	0.6318506	
0.0	0.0778	0.7782220	0.6276163	
0.0	0.0889	0.7782220	0.6272669	
0.0	0.1000	0.7801732	0.6286061	
0.5	0.0001	0.7830859	0.6485584	
0.5	0.0112	0.7850466	0.6498765	
0.5	0.0223	0.7782029	0.6363282	
0.5	0.0334	0.7791928	0.6346189	
0.5	0.0445	0.7801828	0.6343700	
0.5	0.0556	0.7752808	0.6233556	
0.5	0.0667	0.7801637	0.6290569	
0.5	0.0778	0.7821245	0.6304227	
0.5	0.0889	0.7762517	0.6183159	
0.5	0.1000	0.7694270	0.6039675	
1.0	0.0001	0.7840567	0.6503101	
1.0	0.0112	0.7840567	0.6477733	
1.0	0.0223	0.7791833	0.6369605	
1.0	0.0334	0.7733771	0.6250427	
1.0	0.0445	0.7694555	0.6153173	
1.0	0.0556	0.7743290	0.6196923	
1.0	0.0667	0.7713973	0.6108791	
1.0	0.0778	0.7596802	0.5865176	
1.0	0.0889	0.7382067	0.5418499	
1.0	0.1000	0.7186750	0.5008750	

Accuracy was used to select the optimal model using the largest value.
The final values used for the model were $\alpha = 0.5$ and $\lambda = 0.0112$.

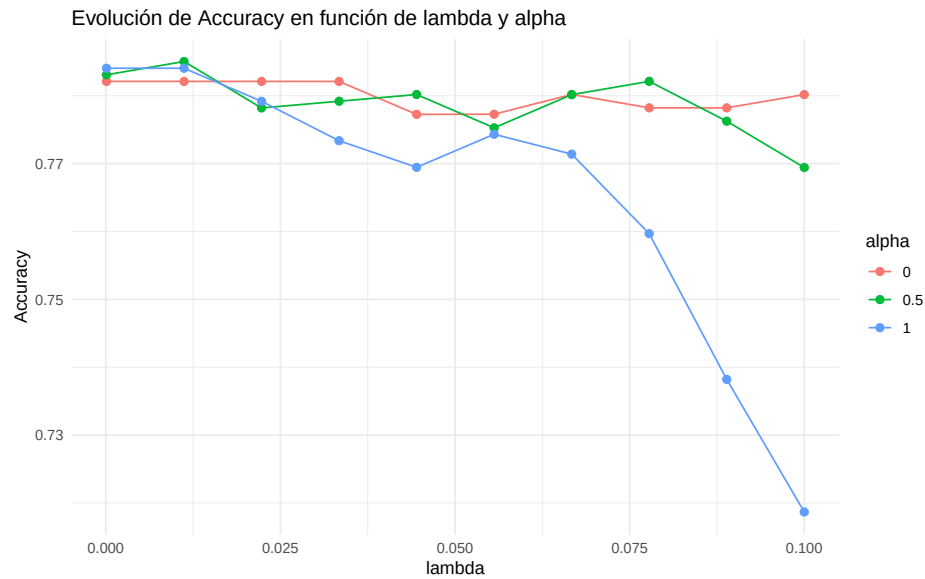
Mejor combinación de hiperparámetros:

Table 25: Mejor Combinación de Hiperparámetros

	alpha	lambda
12	0.5	0.0112

Table 26: Matriz de Confusión - Modelo Tuneado

Prediction	Reference	Freq
Economicas	Economicas	84
Intermedias	Economicas	25
Caras	Economicas	0
Economicas	Intermedias	20
Intermedias	Intermedias	181
Caras	Intermedias	18
Economicas	Caras	0
Intermedias	Caras	24
Caras	Caras	84



Se entrenó un modelo multiclase con regularización , variando y .

La combinación óptima fue $\alpha = 0.5$ $\lambda = 0.0112$ (intermedio entre Ridge y Lasso) y $\alpha = 0.0112$.

Al imponer penalizaciones, se logra reducir el sobreajuste, pero no siempre aumentar la exactitud en validación.

Este modelo obtiene ~80.0% de accuracy, que es ligeramente inferior a los 80.7% de los dos modelos multinomiales; sin embargo, la diferencia no parece estadísticamente grande.

Modelo regularizado

Table 27: Mejor combinación de hiperparámetros

	alpha	lambda
12	0.5	0.0112

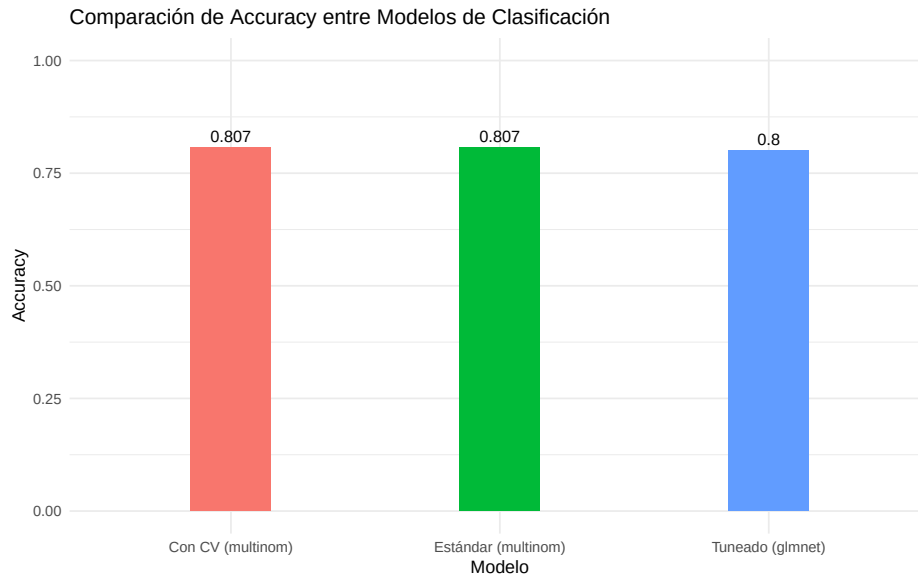
Table 28: Matriz de Confusión - Modelo Regularizado

Prediction	Reference	Freq
Economicas	Economicas	84
Intermedias	Economicas	25
Caras	Economicas	0
Economicas	Intermedias	20
Intermedias	Intermedias	181
Caras	Intermedias	18
Economicas	Caras	0
Intermedias	Caras	24
Caras	Caras	84

Comparación de modelos

Table 29: Resumen de Métricas de los Modelos

Modelo	Accuracy	Kappa	AIC	BIC
Estándar (multinom)	0.8073394	0.6911739	1031.368	1070.820
Con CV (multinom)	0.8073394	0.6911739	1039.339	1078.791
Tuneado (glmnet)	0.8004587	0.6768279	NA	NA



Los dos modelos multinomiales (Estándar y con CV) tienen la misma exactitud (0.807).

La validación cruzada no mejoró la métrica de manera apreciable en este conjunto (posiblemente porque el dataset o la complejidad del modelo no facilitan un mayor incremento en la accuracy).

El Kappa también es idéntico (~ 0.691), lo que sugiere igual poder de discriminación entre clases.

El modelo “Tuneado” alcanza ~ 0.800 de accuracy con un Kappa de ~ 0.677 .

Es levemente inferior en ambas métricas frente a los dos modelos multinomiales.

Esto se debe, en parte, a la penalización introducida por α y λ , que reduce la magnitud de los coeficientes pero no siempre conlleva un mejor ajuste, sobre todo si la muestra no es muy grande o si las variables elegidas ya son lo suficientemente parsimoniosas. ##### Resumen de Coeficientes Ambos modelos multinomiales arrojan coeficientes similares:

- (Intercept): Ajusta el “sesgo” para cada categoría (Intermedias, Caras) en relación con la categoría de referencia (Economicas).
- OverallQual, GrLivArea, YearBuilt: Todas se muestran con coeficientes positivos (, lo que indica que al aumentar la calidad, la superficie habitable o la reciente construcción, crece la probabilidad de ser Intermedias o Caras .

El AIC y el BIC del modelo Estándar son algo menores que los del modelo con CV , lo que sugiere ligeramente que el modelo Estándar podría ser más parsimonioso según estos criterios. De todos modos, la diferencia es pequeña.

Modelo Tuneado:

Se entrenó un modelo multiclase con regularización (Elastic Net), variando α y λ .

La combinación óptima fue $\alpha = 0.5$ $\lambda = 0.0112$ (intermedio entre Ridge y Lasso) y $\alpha = 0.0112$ $\lambda = 0.0112$.

Al imponer penalizaciones, se logra reducir el sobreajuste, pero no siempre aumentar la exactitud en validación (comparado con los modelos sin penalización).

Este modelo obtiene ~80.0% de accuracy, que es ligeramente inferior a los 80.7% de los dos modelos multinomiales; sin embargo, la diferencia no parece estadísticamente grande.

Mátrices de confusión:

Errores comunes en todas las matrices: -

- Un número bajo de confusiones entre “Economicas” vs. “Caras” .
- Más errores entre “Economicas” e “Intermedias”, y también entre “Intermedias” y “Caras”.
- Esto coincide con la intuición de que “Intermedias” a veces se confunde con las otras dos categorías más extremas.

Modelos Estándar y con CV:

- Tienen prácticamente la misma estructura de confusiones, confirmando que los coeficientes y la performance global son muy parecidos.

Modelo Tuneado:

Muestra una redistribución leve de aciertos/errores: por ejemplo, hay algo más de confusiones “Intermedias” “Economicas”. Estas diferencias explican la ligera reducción de accuracy.

Comparacion de Modelos de Clasificación con diferentes algoritmos

weights: 15 (8 variable)

initial value 1124.978984 iter 10 value 509.675531 final value 507.683971 converged

Table 30: Comparación de métricas de clasificación

Model	Accuracy	Kappa	F1
Tree Base	1.0000000	1.0000000	1.0000000
Random Forest	1.0000000	1.0000000	1.0000000
Naive Bayes	0.8967890	0.8403398	0.8978990
KNN	0.9243119	0.8757685	0.9191502
Logistic Regression	0.8073394	0.6911739	0.8070408
Tuned (glmnet)	0.8004587	0.6768279	0.7983228

weights: 15 (8 variable)

initial value 1124.978984 iter 10 value 509.675531 final value 507.683971 converged

Table 31: Comparación de Tiempos de Ejecución de Modelos de Clasificación (en segundos)

Model	Elapsed_Time_sec
Tree (rpart)	0.0800
Random Forest	2.9100
Naive Bayes	0.0500
KNN (Prediction)	0.0600
Logistic Regression	0.0100

- **Eficiencia de Prediccion**

- **Modelos Basados en Árboly Random Forest:**

Estos modelos muestran métricas perfectas .Este resultado puede darse dado que el problema es relativamente sencillo. Sucede lo mismo con KNN y NB puede existir una separación alta de clases o una normalización alta de el conjunto de datos.

- **Modelos Basados en Regresión Logística:**

Tanto el modelo estándar de regresión logística como su versión tuneada con regularización muestran resultados significativamente inferiores. Esto indica que, en comparación con los modelos basados en árboles y vecinos, los modelos lineales son menos precisos para este problema. La menor exactitud y Kappa se traduce en que se cometen más errores al clasificar y, en definitiva, se equivoca en mayor proporción.

- Es importante notar que dada la solución simplificada que estos realizan al problema contribuye a la precisión de los modelos de regresión. Siendo estos bastante buenos para predecir.

El modelo que más aciertos, tomando en cuenta la forma en la que resuelven el problema los algoritmos previos como random forest es KNN, teniendo buenas metricas en general. Una segunda buena opción es Naive Bayes.

El modelo que peor lo hace es el modelo de regresión con ajuste de parámetros. Una posible explicación como se menciona previamente es el sacrificio que se hace al ajustar parámetros.

- **Eficiencia de Tiempo**

- **Eficiencia computacional:**

Los modelos como la Regresión Logística y Naive Bayes son extremadamente rápidos

– **Trade-off entre complejidad y tiempo:**

Aunque Random Forest puede ofrecer ventajas en términos de precisión y robustez al trabajar con datos complejos, su mayor tiempo de ejecución es una desventaja en entornos donde el tiempo de respuesta es crítico.

El que más se demora es Random forest, al evaluar y hacer multiples arboles para predecir. En contraste, la regresión logística tiene el mejor desempeño.

La elección se reduce entre **KNN**, **Naive Bayes** y **Random Forest**.

En la práctica, **Random Forest** resulta el mejor para predecir gracias a su alta exactitud y capacidad para modelar relaciones complejas, aunque se debe tener cuidado con el potencial sobreajuste.

Por ello, elegir **KNN** o **Naive Bayes** no es una mala opción..